Expt. No. 10

Title: *Design and implement a LSTM model for sentiment analysis*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 22/09/2025

Sign: _____

Experiment 10

Aim: Design and implement LSTM model for sentiment analysis

Software: Google Collab

Pre Lab Question:

Explain the CNN architecture?

CNN architecture consists of convolutional, pooling, and fully connected layers for automatic feature extraction and classification.

Post Lab Question:

Explain relation between input, output and filter size?

In CNNs, output size depends on input size, filter size, stride, and padding during convolution operations.

Theory: The core idea of LSTM's are the cell state, and it's various gates. The cell state act as a transport highway that transfers relative information all the way down the sequence chain. You can think of it as the "memory" of the network. The cell state, in theory, can carry relevant information throughout the processing of the sequence. So even information from the earlier time steps can make it's way to later time steps, reducing the effects of short-term memory. As the cell state goes on its journey, information get's added or removed to the cell state via gates. The gates are different neural networks that decide which information is allowed on the cell state. The gates can learn what information is relevant to keep orforget during training.

Technically, LSTM inputs can only understand real numbers. A way to convert symbol to number is to assign a unique integer to each symbol based on frequency of occurrence. For example, there are 112 unique symbols in the text above. The function in Listing 2 builds a dictionary with the following entries ["," : 0] ["the" : 1], …, ["council" : 37],…,["spoke" : 111]. The reverse dictionary is also generated since it will be used in decoding the output of LSTM.LSTMs are primarily used in NLP and Time series forecasting. LSTM is really powerful that the results achieved using RNN's can also be achieved using LSTM.

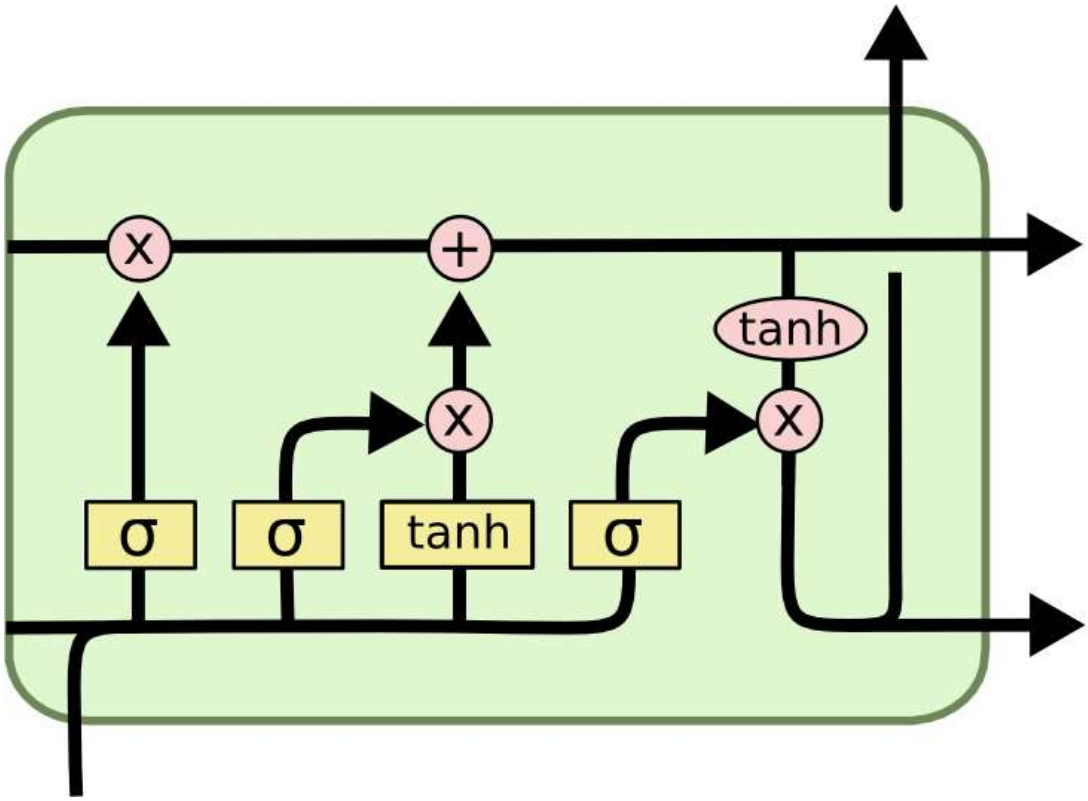
Sentiment analysis is the classification of emotions (positive, negative, and neutral) within data using text analysis techniques. Harnessing the power of deep learning, sentiment analysis mod els can be trained to understand text beyond simple definitions, read for context, sarcasm, etc., and understand the actual mood and feeling of the writer.

Architecture of LSTM

STMs deal with both Long Term Memory (LTM) and Short Term Memory (STM) and for making

thecalculations simple and effective it uses the concept of gates.

1. **Forget Gate:** LTM goes to forget gate and it forgets information that is not useful.
2. **Learn Gate:** Event (current input) and STM are combined together so that necessary information that we have recently learned from STM can be applied to the current input.
3. **Remember Gate:** LTM information that we haven't forget and STM and Event are combined together in Remember gate which works as updated LTM.
4. **Use Gate:** This gate also uses LTM, STM, and Event to predict the output of the current event which works as an updated STM.



Steps for performing Sentiment analysis using LSTM

Step 1) Importing the necessary libraries

Step 2) Read and train /test files using pandas

Step 3) Perform data pre-processing

Step 4) Import the API's from tensor flow to build LSTM model.

Step 5) Perform early stopping and save model checkpoints.

Step 6) Generate the graph function

Step 7) Define the prediction function

Step 8) Summarise the build model

Step 9) Predict the results after training the results.

Program & Output

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.datasets import imdb

# 1) Load IMDB dataset (already labeled)
vocab_size = 10000
maxlen = 100  # max number of words per review

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

# 2) Pad sequences to ensure equal length
X_train_pad = pad_sequences(X_train, maxlen=maxlen, padding='post', truncating='post')
X_test_pad = pad_sequences(X_test, maxlen=maxlen, padding='post', truncating='post')

# 3) Build LSTM model
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=128, input_length=maxlen),
    LSTM(64, return_sequences=False),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()

# 4) Add EarlyStopping and Checkpoint
es = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
mc = ModelCheckpoint('best_imdb_lstm_model.h5', monitor='val_loss',
save_best_only=True)

# 5) Train the model
history = model.fit(
    X_train_pad, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2,
    callbacks=[es, mc]
)
```

```
# 6) Evaluate on test data
loss, acc = model.evaluate(X_test_pad, y_test)
print(f'Test Loss: {loss:.4f}, Test Accuracy: {acc:.4f}')


# Prediction

# 7) Word Index to convert raw text
word_index = imdb.get_word_index()

def encode_text(text):
    tokens = text.lower().split()
    encoded = [1]  # start token
    for word in tokens:
        encoded.append(word_index.get(word, 2))  # unknown token if not found
    return pad_sequences([encoded], maxlen=maxlen, padding='post', truncating='post')

def predict_sentiment(text):
    encoded = encode_text(text)
    prediction = model.predict(encoded)[0][0]
    return f"{'Positive' if prediction > 0.5 else 'Negative'} ({prediction:.2f})"

# 8) Try predictions
print(predict_sentiment("This movie was fantastic! I loved every moment of it."))
print(predict_sentiment("Worst movie ever. Don't waste your time."))
```

OUTPUT:

1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 29ms/step
Positive (0.85)
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 29ms/step
Positive (0.85)

Conclusion:

Designing and implementing an LSTM model for text data enables the capture of long-term dependencies and contextual information within sequences. This makes LSTM especially effective for complex natural language tasks like sentiment analysis and text classification.

CO's Covered:

CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.